AI 技術レポート：Antigravity を用いた Tetris プロジェクトの実行と分析

使用した AI サービス：Antigravity

提出用ドラフト

1. （1）やったこと・動かしたものの概要

本レポートでは、AI コーディング支援サービスである **Antigravity** を用いて **Tetris** を **vibecoding** し、その過程を整理した。対象となるプログラムは React、TypeScript、Vite を用いて作られたブラウザ版での Tetris である。

今回の目的は、**Antigravity を用いて Vibecoding を行い、その有用性と限界を確認すること**である。具体的には、次の作業を行った。

- Antigravity に Tetris を生成する指示を与える
- 生成されたゲームに対して、微調整を行う
- 以上の過程を、複数回繰り返しデバグや追加機能を入れた。

最終的には以下の機能まで実装が行えた。

- テトリミノの落下
- 左右移動
- 回転
- ライン消去
- スコア、行数、レベルの更新
- Hold 機能
- キーによるミノの変更 (チート mode)

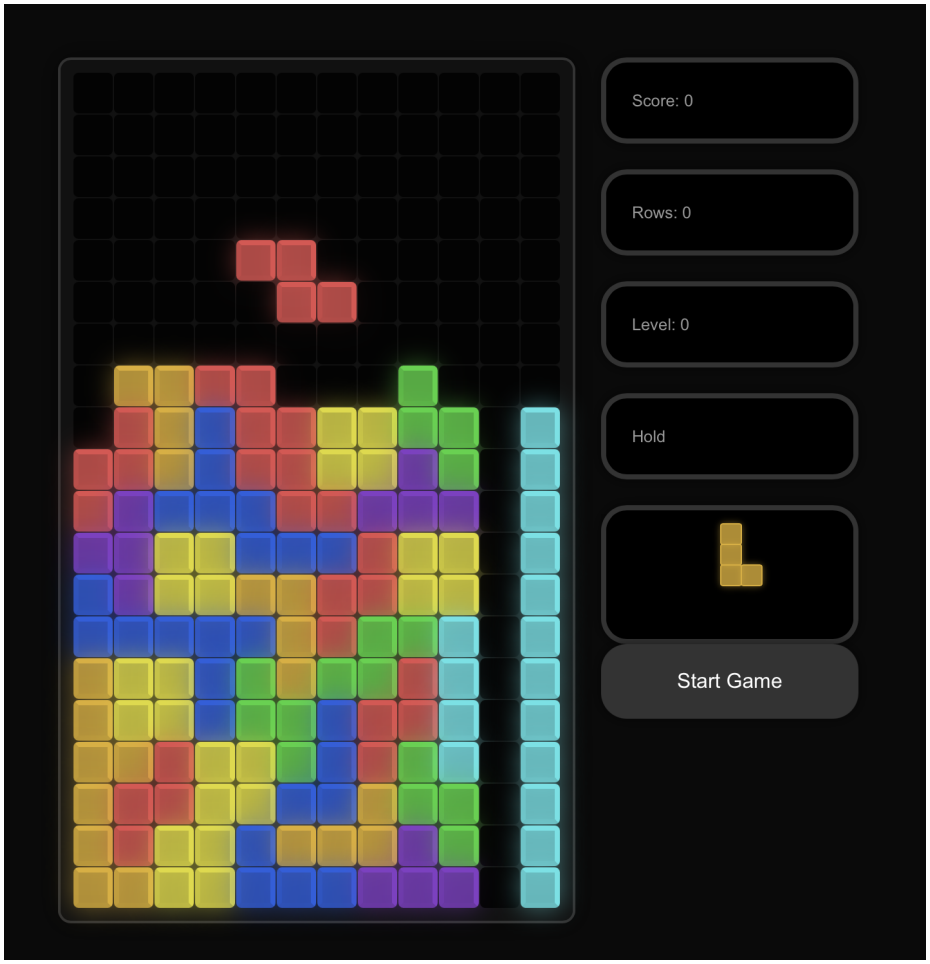

図 1　作成したテトリスの画像

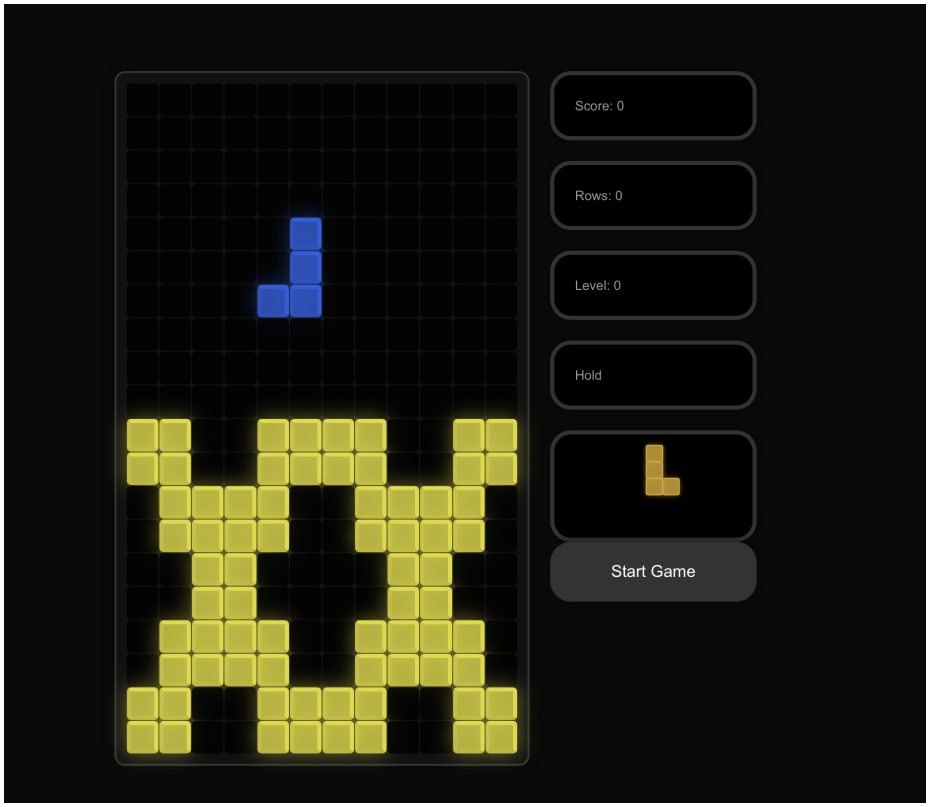

図 2　テトリスですべて o ミノにして不正した画像

2.　（2）具体的な手続き

2.1. 実行手順

実際に行った手順は以下の通りである。

1. プロジェクトディレクトリを作成
2. Antigravity の起動
3. 自然言語のみで、Antigravity　に Tetris を生成するよう伝える (1)
4. 複数回デバグを行う
5. 追加機能としてキー入力で任意のミノに変更できるようにした

2.2. 主なデバグ内容

1. 一番最初の指示で綺麗に生成することはできなかった。
2. 主な原因は左右の壁に当たるとその場で止まってしまう。
3. 左右の壁が繋がってしまい、左右が通り抜けることができる。

3. （3）評価・考察

3.1. Antigravity のメリット

1. 知らない言語に対して、実行したいものを瞬時に作成ができる。
2. １時間程度でゲームが作成できたため、最低限のコードエージェントとして活用できる。
3. デバグしてほしい内容に対して適切にデバグできた。

3.2. Antigravity のデメリット

1. 内部の構成がわからないため、セキュリティなどには不安が残る
2. 最初から完璧に作ることはできなかった
3. デバグの量もかなりあった。

3.3. 今後の検証候補

1. より高度な検証をするため、より複雑なゲームの生成を試みる。
2. 同じゲームを codex や claude にも書かせることでモデルや Agent の比較を行う
3. プロンプトの設定による性能の変化を検証する。

4. まとめ

本レポートでは、AI 技術として **Antigravity** を用い、その実行後に得られた React + TypeScript 製の Tetris プロジェクトを生成した。

その結果、Antigravity のような AI コーディング支援は、短時間でアプリケーションの土台を得る手段として有用である一方、生成結果をそのまま提出できるとは限らず、**人間が実行して確認し、必要な修正を加えることが不可欠である**と分かった。

今回の体験を通じて、AI は「完全自動で完成品を出すもの」というより、**開発の初速を上げる補助ツール**として使うのが現実的であると感じた。

付録：提出確認に用いたコマンド

```
npm run build
```